Dataset and Preprocessing:

We use the train_with_task_type.csv from the CoderGym Nemotron annotation, which aligns with the Nemotron prompts and adds human-readable labels. The cleaning that we had to do is to strip the whitespace from the column names and drop rows that are missing prompt or task_type. We use a 80/20 train/test split and 5-fold stratified cross-validation.

Pipeline::

The pipelines that we implemented are the TF-IDF + Linear SVM, TF-IDF + Naive Bayes, and Word2Vec + RandomForest. The other models compared were KNN, trees, and neural networks, but not part of the comparison.

Evaluation Metrics:

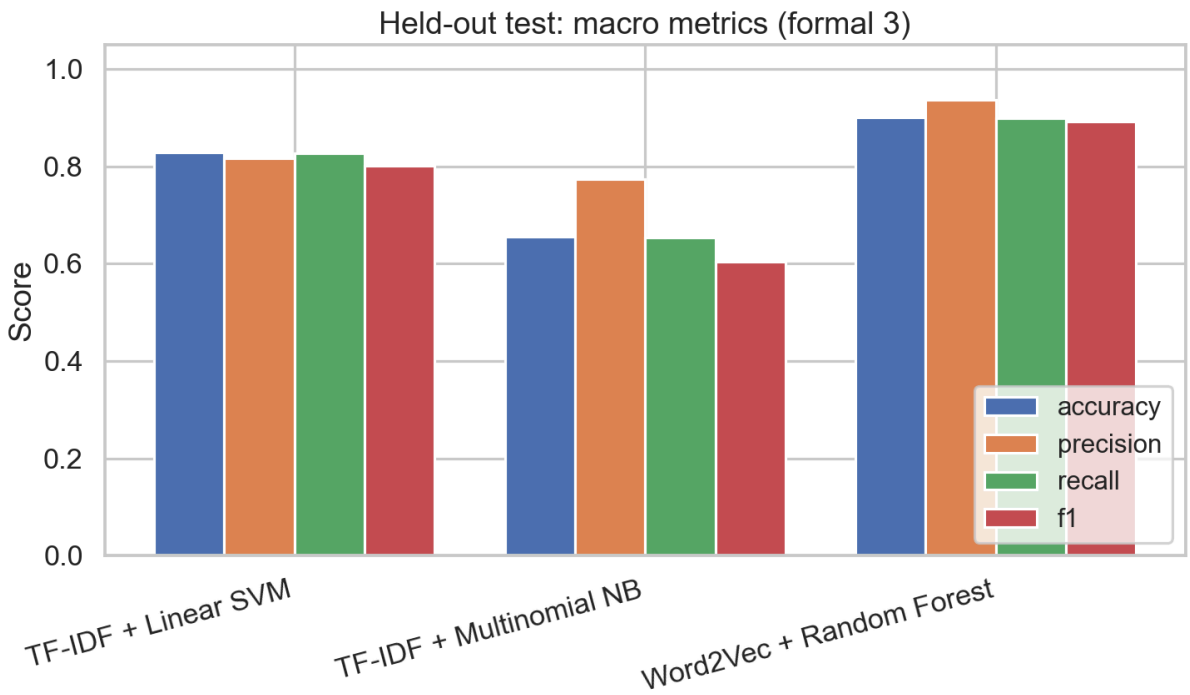
Pipeline	**Accuracy**	**Macro F1**	**CV Macro F1 (mean ± std)**	**Train Time (s)**	**Inference Time (s)**
TF-IDF + Linear SVM	0.8284	0.8010	0.8041 ± 0.0042	0.160	0.020
TF-IDF + Naive Bayes	0.6563	0.6048	0.6097 ± 0.0084	0.107	0.023
Word2Vec + Random Forest	0.9000	0.8921	0.8989 ± 0.0095	1.998	0.259

Word2Vec + RandomForest achieved the highest accuracy and F1-score but took longer to train. TF-IDF + SVM has strong performance but faster speed. Naive Bayes is the fastest but the least accurate.

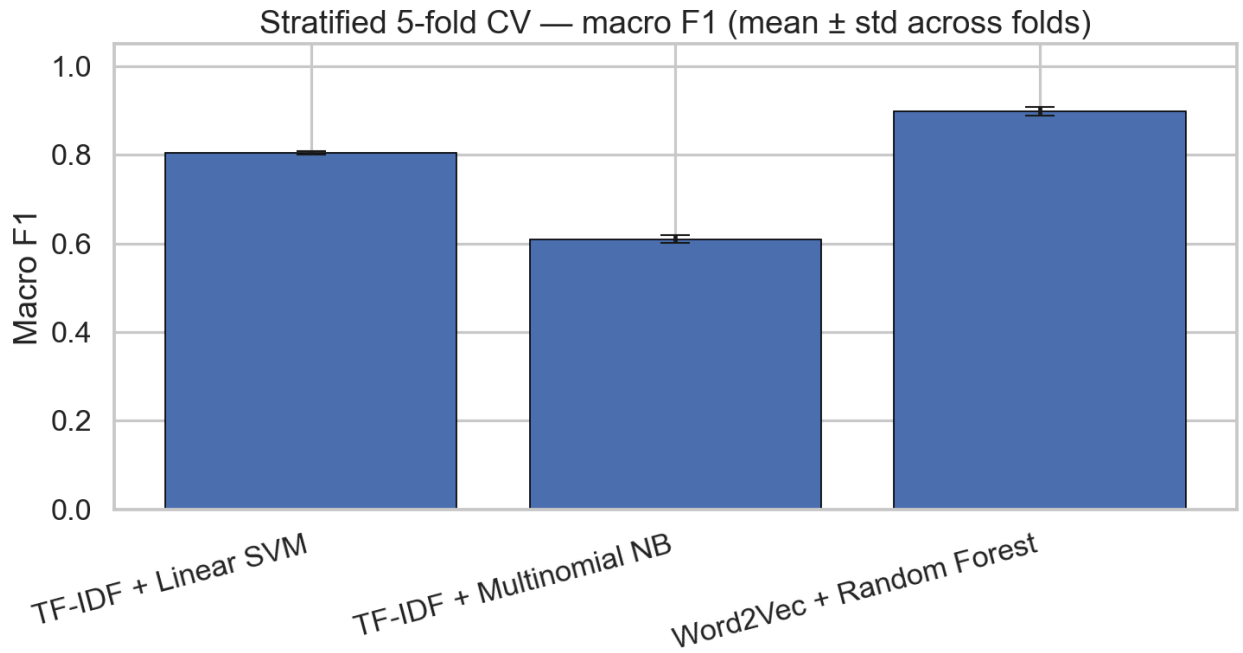
Comparison Figures

Figures are in the /figures folder and are created when you run train_evaluate.py.

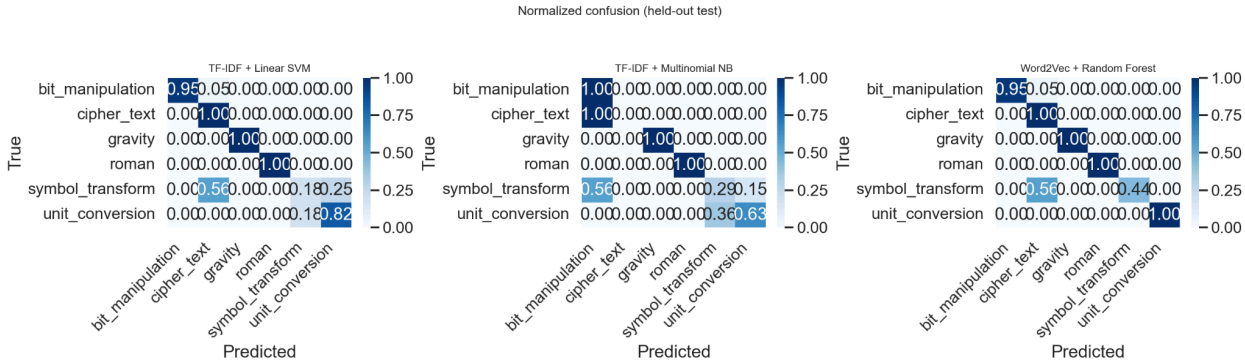
Formal three — Hold-out metrics



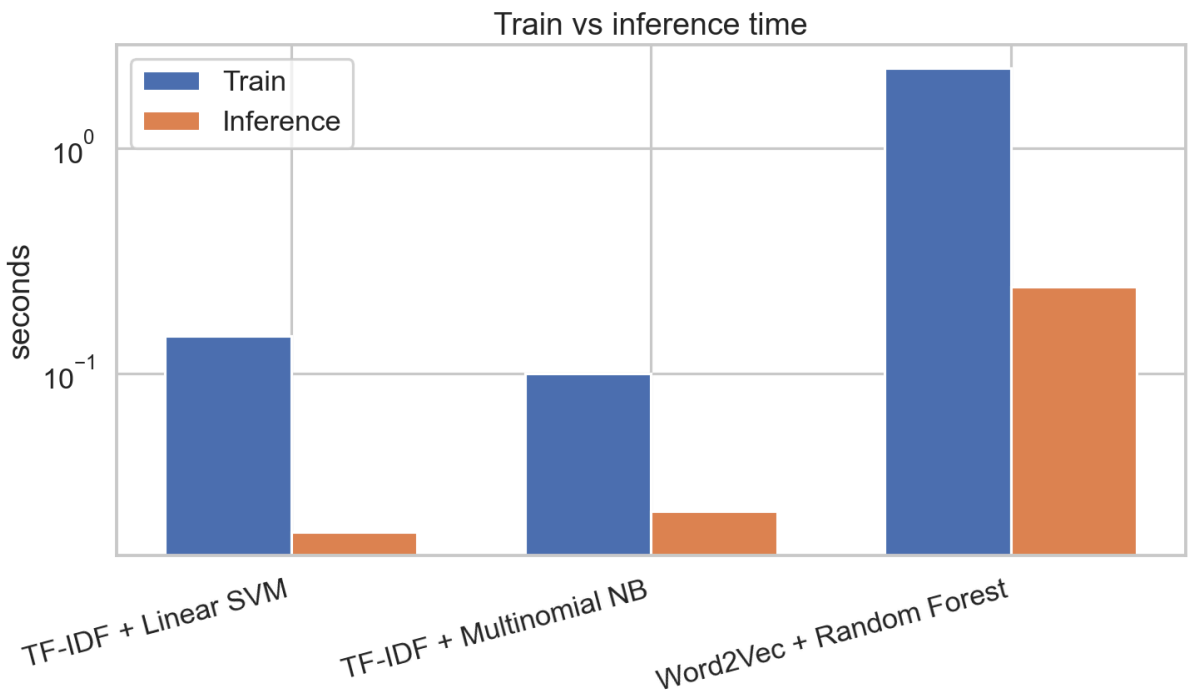
Cross-validation macro F1



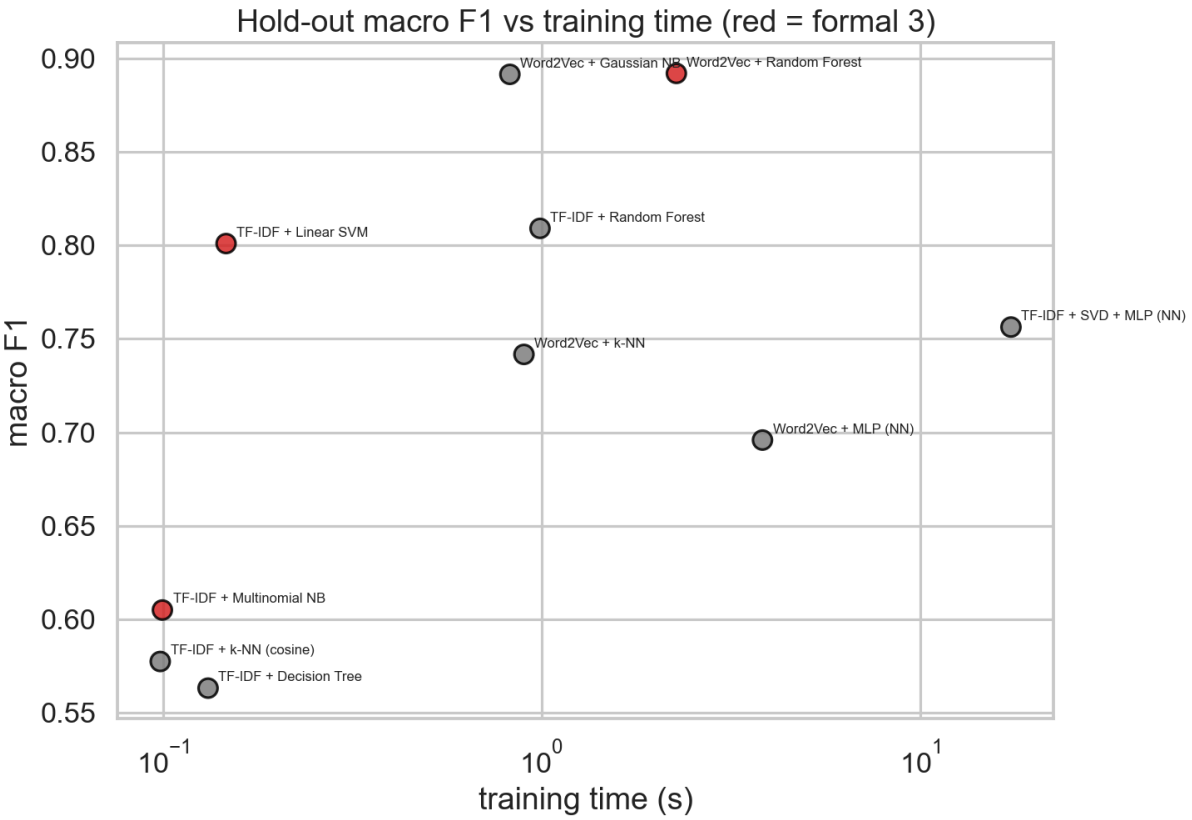
Confusion matrices



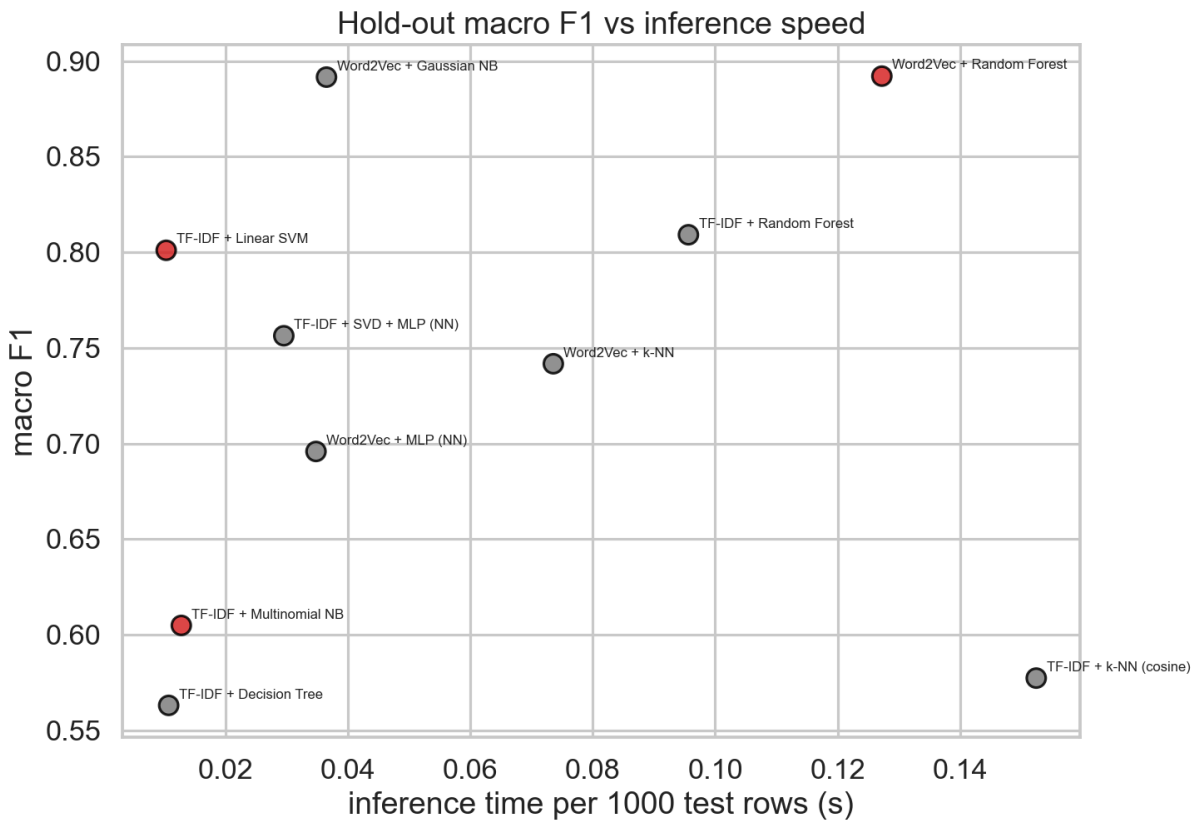
Training vs inference time



Performance vs training time (all pipelines)



Performance vs inference time (all pipelines)



Recommendations

So we determined Word2Vec + Random Forest has the highest accuracy and F1-score but slower training time. TF-IDF + SVM has strong performance but faster speed. Naive Bayes is the fastest but the least accurate. Hence, the clear trade off here is the higher accuracy requires more computing. The recommendation I would take here is the TF-IDF + Linear SVM due to its balance of performance in accuracy and the faster training than Random Forest.